WEB COMPONENTS NATIVOS

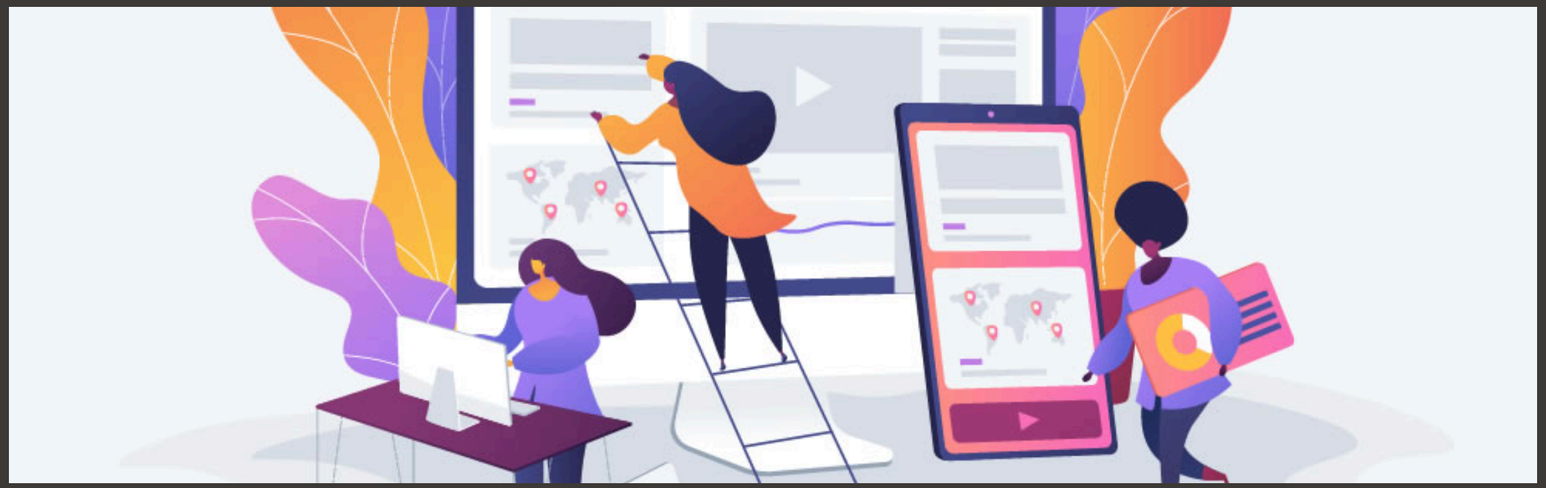
Custom Elements, Shadow DOM y HTML Templates

¿QUÉ SON LOS WEB COMPONENTS?

Definición: Un conjunto de API nativas de la plataforma web que permiten crear etiquetas HTML personalizadas, reutilizables y encapsuladas.

Los 3 Pilares Fundamentales:

1. Custom Elements: Permiten definir nuevas etiquetas HTML y controlar su comportamiento con JavaScript.
2. Shadow DOM: Aisla el código CSS y HTML dentro del componente para que los estilos externos no lo rompan (y viceversa).
3. HTML Templates (<template> y <slot>): Plantillas de código que no se renderizan hasta que las usas con JavaScript, ideales para estructurar el componente.

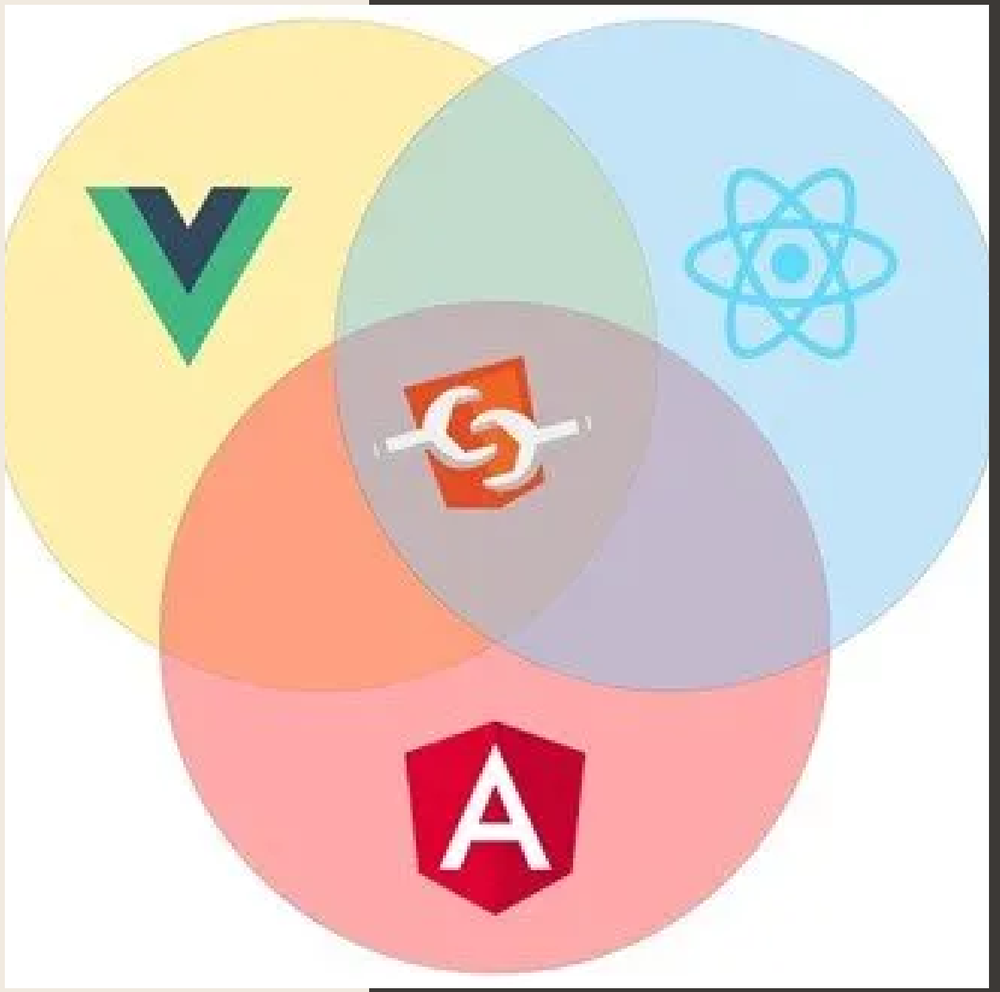


HISTORIA RÁPIDA

- Origen (2011): Alex Russell (Google) propuso la idea por primera vez para estandarizar cómo se crean componentes en la web.

- Web Components v0 (2014): Primera versión experimental, adoptada principalmente por Chrome, pero no tuvo acuerdo entre los demás navegadores.

- Web Components v1 (2016 - Presente): Se rediseñó el estándar con el consenso de Apple, Mozilla, Google y Microsoft. Es la versión oficial que todos los navegadores soportan hoy en día.



¿QUÉ TANTO SE USAN?

EN GRANDES EMPRESAS:

Las usan compañías como Google (YouTube está hecho en gran parte con Web Components), Adobe, Salesforce, SpaceX y GitHub.

USO EN PROYECTOS:

Son muy populares para crear Design Systems (librerías de UI internas) que deben funcionar en múltiples proyectos sin importar si usan React, Angular o HTML plano.

VENTAJAS Y DESVENTAJAS

VENTAJAS

SIN DEPENDENCIAS:

No necesitas npm install ni librerías externas.

COMPATIBLES

Funcionan en cualquier framework o con HTML/JS puro.

ENCAPSULAMIENTO REAL

El Shadow DOM evita conflictos de estilos CSS.

DESVENTAJAS

SINTAXIS ALGO LARGO:

Requieren escribir más código JavaScript para cosas simples.

FORMULARIOS NATIVOS:

Integrar componentes personalizados dentro de un <form> tradicional requiere código extra .

¿PARA QUÉ SIRVEN?

- Crear librerías de UI universales: Diseñar botones, tarjetas o modales que sirvan para cualquier equipo de desarrollo.
- Micro-frontends: Integrar partes de una aplicación web construida con distintas tecnologías.
- Reducir peso en la web: Al no incluir un runtime de un framework, las aplicaciones cargan más rápido.



CONCLUSIÓN

- Son la base estándar de la web para la modularidad.
- Ideales cuando buscas rendimiento nativo y compatibilidad a largo plazo.